Hazard Analysis
Keystone

Team 13

Ayush Patel
Ahmed Rashrash
Andrew Habib
Waqar Ul-Hassan

Table 1: Revision History

Date	**Developer(s)**	**Change**
Oct 18th	Ayush Patel	Introduction, Scope, System Boundary and Assumptions

Contents

1 Introduction **1**

2 Scope and Purpose of Hazard Analysis **1**

3 System Boundaries and Components **2**
3.1 System Boundary 2
3.2 Major Components 3

4 Critical Assumptions **4**

5 Failure Mode and Effect Analysis **5**

6 Safety and Security Requirements **5**

7 Roadmap **5**

8 Appendix — Generative AI Use Disclosure **6**
8.1 AI Tools Used 6
8.2 How and Where Used 6
8.3 Verification of AI-Generated Content 6

9 Appendix — Reflection **7**

1 Introduction

Keystone is a capstone project discovery and management platform intended to support the lifecycle of a capstone project, from project proposal and discovery through team formation, project approval, and the management of project information. The platform is intended to serve students, project supervisors, course instructors, and teaching assistants (TAs).

Although Keystone is not intended to control physical equipment or perform safety-critical operations, failures can still cause harm. Examples include unauthorized disclosure of confidential project and student information, invalid team assignments, lost or inconsistent project updates, and misleading approval or notification status. These failures could delay academic activities, create unnecessary work, and damage trust.

We define a hazard as an unsafe system condition that, in combination with particular environmental circumstances, can contribute to an undesirable outcome. Hazard analysis examines how system failures and interactions can produce such conditions and identifies measures to eliminate or control them (**??**).

This document identifies potential hazards associated with Keystone, analyzes the ways in which system components may fail, and proposes mitigations that can be translated into verifiable safety and security requirements.

2 Scope and Purpose of Hazard Analysis

The scope includes functional failures, integration failures, security and privacy weaknesses, usability and accessibility issues, and foreseeable user errors. It also considers failures at the boundaries between Keystone and external services.

This hazard analysis covers the following categories of loss:

- **Disruption of academic activities:** Failure to discover projects, form eligible teams, obtain approvals, or maintain accurate project records may delay project selection and interfere with course milestones.
- **Unauthorized disclosure or misuse of information:** Confidential project proposals, intellectual property concerns, contact information, student profiles, and team records may be exposed to people who are not authorized to access them.
- **Incorrect academic or administrative decisions:** Inaccurate project descriptions, invalid team assignments, incorrect course rules, or misleading approval statuses may cause students, supervisors, or instructors to make decisions based on incorrect information.
- **Loss or corruption of project information:** Failed updates, integration errors, or inconsistent records may cause project details, supervisor information, repository links, or selected documentation requirements to become unavailable or outdated.

- **Loss of trust and adoption:** Unreliable notifications, unclear workflows, inaccessible interfaces, or inappropriate disclosure of information may frustrate users and reduce confidence in the platform.

3 System Boundaries and Components

3.1 System Boundary

Keystone is the system under analysis. Its boundary includes the application logic, persistent data, user-facing functionality, and integrations implemented or configured specifically for the platform. See Figure 1

External services are not assumed to be under Keystone's direct control. Nevertheless, their interfaces and the consequences of their failure are included in the analysis because Keystone depends on them to provide its intended functionality.

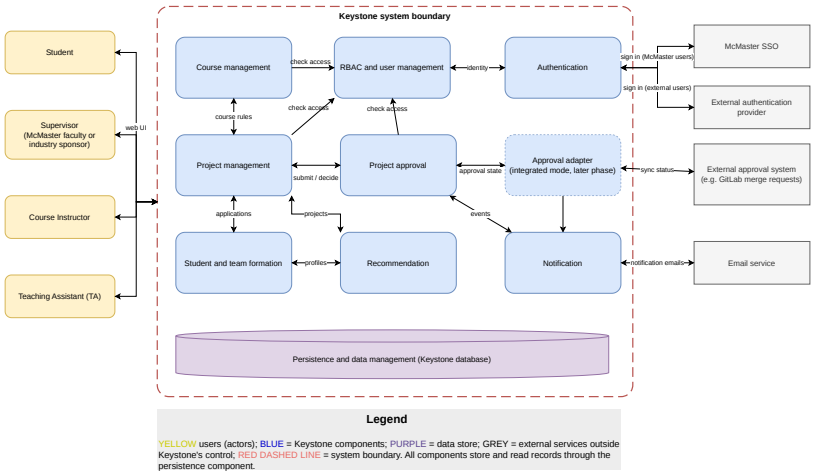


Figure 1: System and boundary context diagram for Keystone

The following external entities and services are relevant:

- **McMaster single sign-on (SSO):** Provides authentication for users accessing Keystone through their McMaster credentials.
- **External authentication provider:** Supports authentication for users who do not have a McMaster account.
- **Email service:** Delivers notifications concerning project submissions, approvals, supervisor confirmations, and other events.

- **External approval system:** A course-specific system outside Keystone where a course approves team projects, for example the GitLab merge request process used by the Software Engineering course. It is not under Keystone's control. In *external* approval mode, the instructor or a TA records the approval outcome in Keystone by hand. In *integrated* mode, an approval adapter (a Keystone component) reads the approval state through the external approval system's interface and updates Keystone's records.

3.2 Major Components

The following components define the principal areas in which failures will be investigated. They are meant to be functional boundaries for hazard identification, rather than a complete software architecture.

1. **Authentication component:** Manages user sign-in through McMaster SSO and an external authentication provider. It is responsible for establishing the identity associated with a user session. Authentication failures may allow unauthorized access or prevent legitimate users from accessing the platform.

2. **Role-Based Access Control (RBAC) and user management component:** Maintains user accounts, roles, and permissions. It determines which resources users may view or modify. Relevant roles include students, supervisors, TAs, and course instructors. Incorrect permissions may expose restricted information or allow unauthorized changes.

3. **Course management component:** Maintains course configurations, course membership, instructor and TA assignments, course-specific rules and configurations, and relevant deadlines or course information. It must accommodate differences between Software Engineering (SE), Computer Science (CS), and Mechatronics (TRON) courses.

4. **Project management component:** Manages project proposals, descriptions, supervisors, contact details, project categories, confidentiality information, intellectual property (IP) concerns, repository links, and project status throughout the project lifecycle.

5. **Project approval component:** Supports project proposal review, team project approval, and the recording of approval decisions. It must distinguish between submission, review, approval, and other workflow states so that users are not misled about whether a project or team has been approved.

6. **Student and team formation component:** Manages student profiles, team membership, project interests, and any supported team or project recommendations. It must respect course eligibility rules and team-size constraints.

7. **Notification component:** Generates and sends notifications associated with relevant workflow events via the email service. Failures may result in missing, delayed, duplicated, or incorrectly addressed messages.

8. **Persistence and data management component:** Stores and retrieves the platform's persistent records. It maintains the consistency of related information, including project records, team membership, course configuration, approval states, and user permissions.

9. **Approval adapter (integrated mode):** Reads approval state from an external approval system and updates Keystone's records. Failures may leave Keystone showing a state that differs from the external system.

10. **Recommendation component:** Ranks or filters projects and potential team matches based on available information and user preferences. Incorrect recommendations may hide relevant opportunities or mislead users about their suitability for a project.

4 Critical Assumptions

1. **Instructor authority:** Course instructors remain responsible for final project approval, the interpretation of course rules, and exceptional team-formation decisions. Keystone supports these decisions but does not replace the instructor's judgment regarding project feasibility, scope, or academic suitability.

2. **Authentication provider availability:** McMaster SSO and any external authentication provider are external dependencies and may become unavailable. Keystone cannot guarantee their continuous operation but must handle authentication failures safely.

3. **Accuracy of user-supplied information:** Students and supervisors may submit incomplete, incorrect, misleading, or outdated information. The platform can validate information according to defined rules, but it cannot guarantee the truth of every project description, supervisor commitment, or user-provided statement.

4. **Feature configuration:** Optional features may be disabled by course instructors to support different capstone courses or future deployments. Disabling a feature must not leave dependent workflows in an invalid state or expose information that would otherwise be restricted.

5 Failure Mode and Effect Analysis

6 Safety and Security Requirements

7 Roadmap

8 Appendix — Generative AI Use Disclosure

8.1 AI Tools Used

8.2 How and Where Used

8.3 Verification of AI-Generated Content

9 Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

Waqar:

1. What went well while writing this deliverable? During this deliverable, distributing work was very straightforward as there was a lot of work, but it was separable by section. It was all work we have seen before in prior courses with documentation, so it was very intuitive to write and come up with solutions/answers.

2. What pain points did you experience during this deliverable, and how did you resolve them? During this deliverable, everyone was very busy because we had midterms and a lot of assignments due alongside this report. Sometimes it also became difficult to stay updated with everyone's parts. However, because we planned out our schedule well in advance, we were able to finish everything on time while also completing our other work. Everyone also posted updates in our team chat when they got the chance, so we all stayed in the loop.

3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did they come about? Before this deliverable we had thought about things like security issues or our app crashing, which would cause downtime. While working on the hazards, we came up with how a bug could lead to incorrect decisions being made in the actual workflow of our app, which would be chaotic. We also came up with the idea of security regarding the profiles of each person, as we wanted to collect some information to help supervisors get a better idea of who they are working with (the students).

4. Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider? First, data leakage is always a concern with any software product. It drives whether users can trust the developers. If we have a leak, how we handle it and what

measures we take to prevent it show users that we want to protect them and that using the app is safe. Second, availability matters because the system handles time-sensitive information such as deadlines. We want the greatest uptime we can achieve so users do not have to worry about the app going down. Lastly, data correctness is important: while we are handling PII, we want to process it correctly and avoid sharing it with third parties without user consent.

Ayush:

1. What went well while writing this deliverable?

   A great part about this document is it really made us think about our project from a different perspective. It made us think about all the ways things could go wrong, maybe an external service failed, maybe a user did something wrong. It made us think about making our design more robust so it can handle external failures. It also made us think about internal system failures that could have drastic impacts on functionality, such as RBAC not working properly and data integrity concerns. Overall, it was actually quite intuitive to write this deliverable.

2. What pain points did you experience during this deliverable, and how did you resolve them? The main issues with this deliverable were related to keeping everything consistent with the hazard analysis and the SRS. The scope of the system outlined in the SRS dictates the scope of the HA. The hazards and mitigation strategies outlined in the FMEA table must map directly to either functional, non-functional or data requirements in the SRS. But because everything is constantly changing while we are writing it, it was quite difficult to make sure both documents aligned.

3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did they come about?

   Going in, we mostly had the obvious software risks in our heads. We knew that login and permissions (SSO, external accounts, who can see what) could go wrong. We also worried about privacy via student profiles leaking, and supervisors' or students' project ideas getting "stolen". And we knew that if the platform broke, team formation and approvals would get delayed.

   Most of the more specific risks only showed up once we went component by component through the FMEA. For example, we only thought about hurt feelings and fairness when we went through the HA slides and checklist. Additionally, we had not thought about data retention policies until we properly analyzed the persistence module.

4. Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider?

One risk is security and privacy risk. This covers things like someone getting into an account they shouldn't, or personal info and confidential project ideas leaking. It matters because people trust the platform with their data, and once that trust is gone they stop using it. There can also be legal consequences, since software systems have to follow privacy law.

Psychological and social risk is a big risk in software systems. This is stress, embarrassment, feeling unfairly treated or excluded by how the software behaves. It matters because software shapes how people feel about a process. Something like a blunt rejection message or a visible ranking can really hurt someone, even though nothing "broke".